

GENERATING FORMAL RUNTIME VERIFICATION SPECS FROM NATURAL LANGUAGE

Daniel Feng
TJSTAR

Project Layout

Problem

- RV, FL, MOP, ERE
- Utilizing transformers

Background (Related Work)

- No exact background, but exists for other systems
 - Req2LIL
 - NLSpec
- Dataset of Java MOP specs

Novelty

- Generate specifically for MOP
- Open-source results, test on larger datasets, improved accuracy

Methods

- Prompt engineering (final LLM)
 - Attempted tuning
- NL Clarifier
- Event Interpreter
- Comparison tool that converts to regular expression

Results

- Table of results

Conclusion/Future Work

- Did not receive the best accuracy but shows merit in improving spec generation
- Expand to real-world industry testing
- Attempt to use other FL languages
- Survey More LLMs

Problem

- **Runtime verification (RV)** theory is the process of monitoring a software's runtime execution **trace** (the sequence of events) and verifying that the trace satisfies certain properties.
 - In **natural language**, some properties could be:
 - “Every file opened must be closed”
 - “Never modify a collection when it is being accessed”
- Software engineers use RV to verify program requirements

Example:

Events: [open_door = “Open car door”, close_door = “Close car door”,
insert_key = “Insert car keys”, ignite = “ignite car engine”]

Trace: open_door close_door open_door close_door insert_key ignite

Properties: “Never attempt to open_door when the door is already open”
“Ignite car engine must happen after car keys are inserted”

Problem

- However, computers cannot accept natural language, and it must be translated into some “**formal logic**”, equations and code that rigorously define what properties need to be verified
- In this project, we explore being able to generate ERE, or Extended Regular Expressions, one specific language of these formal logics
- Further, we operate under the MOP framework [11], which is one specific RV framework

Background

- ERE is very similar to Regular Expression

**= Match any amount, += Match at least 1, ()= group events, |= “or” events*

/AB(D|C)/*

AAABBCD

- The main difference is that ERE searches for **event** patterns within execution trace while RE searches for character patterns within text

/create_iter use_iter (destroy_iter|pause_iter)/*

create_iter create_iter create_iter use_iter use_iter pause_iter destroy_iter

- We can throw an exception based on the result of the ERE (match, or fail to match)
- For this project, we will assume the available events have already been defined

Background

- What are events?

```
event remove before(Iterator i) :  
    call(void Iterator+.remove()) && target(i) {}  
  
event next before(Iterator i) :  
    call(* Iterator+.next()) && target(i) {}
```



```
ere : (next+ (remove | epsilon))*
```

- RV frameworks allow us to define what the events mean with code
 - The “remove” event occurs when `Iterator.remove()` is called (`i.remove()`)
 - The “next” event occurs when `Iterator.next()` is called (`i.next()`)
 - Epsilon = Empty event (nothing happens)
- Then, ERE allows us to use the events to capture event patterns in the trace

Background: nl2spec, 2023 [1]

- Works on general RV tasks

A Framework for Translating Unstructured Natural Language to Temporal Logics with Large Language Models

[Home](#) [About](#)

Prompt

Translate this sentence to LTL:
Globally, grant 0 and grant 1 do not hold at the same time until it is allowed.

Model: codex

Prompt: generic

Number of tries: 3

Temperature: 0.20

Subtranslations

[+ Add Subtranslation](#)

[Delete All](#)

Translate globally

to G

66.67%

Translate grant 0

to g0

100%

Translate grant 1

to g1

100%

Final Result

[Translate to LTL](#)

$G((!(g0 \ \& \ g1)) \ U \ a)$

100.0%

Background: nl2spec, 2023 [1]

- **Input:** Natural language (English) describing a Linear Temporal Logic (LTL) formula (in this case the events are grant 1 and grant 0)
 - **Does not take existing events as input**
- **Output:** an LTL expression that matches the natural language

Though I am not working with LTL, similar insights can be made as they are both formal logic languages:

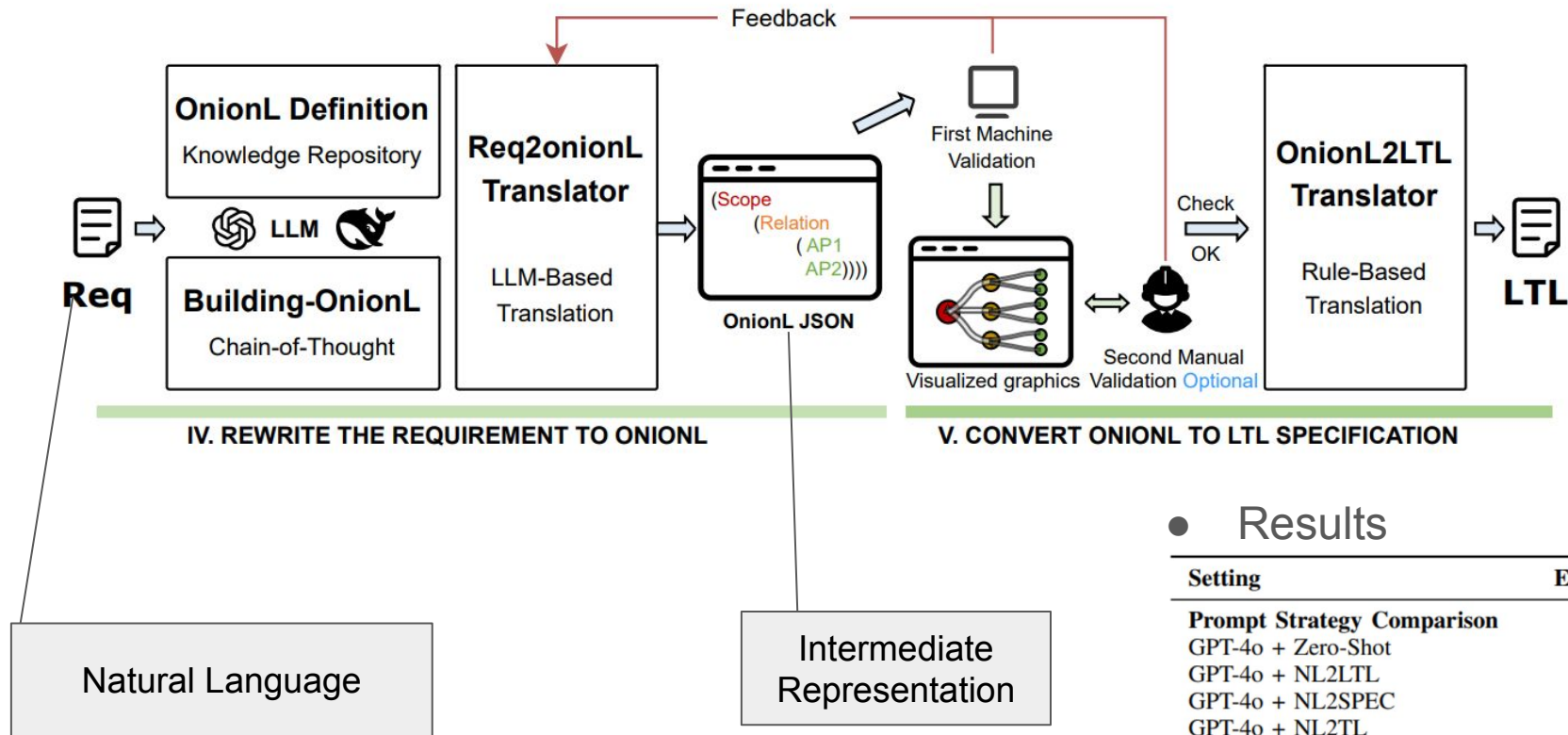
- **Method:** Using Codex model, along with a prompt
- **Accuracy:** 58%; manually verified by field experts

Constraints:

- Very low accuracy without human-intervention
- Prompt used does not follow conventions found in the OpenAI engineering guide
- Codex is fine-tuned on GitHub repos, not special runtime-verification specification repos
- Lack of automatic comparison tool led to small dataset and forced expert comparison

Background: Req2LTL, 2025 [10]

- Used for Aerospace RV specification



Results

Setting	Exact Match (%)
Prompt Strategy Comparison	
GPT-4o + Zero-Shot	43.8
GPT-4o + NL2LTL	55.4
GPT-4o + NL2SPEC	56.3
GPT-4o + NL2TL	65.2
GPT-4o + Req2LTL	88.4

Background: Dataset [3]

- There is no prior work for this exact problem of generating formal languages necessary of Runtime Verification
- There exists a manually curated dataset from researcher Legunsun [3] containing paired natural language descriptions and their corresponding formal specifications.
- The dataset consists of **137** annotated Java API documentation, where NL sentences indicating specification-relevant properties are labeled and used to create formal specifications for runtime verification.

Background: Full NL-RV File Example

Natural Language

Parameters

Events

Logic

Handler

Example from [Iterator RemoveOnce](#) [3]

```
package mop;

import java.util.*;
import java.lang.*;
import com.runtimeverification.rvmonitor.java.rt.RVMLogging;
import com.runtimeverification.rvmonitor.java.rt.RVMLogging.Level;
```

```
/**
 * Warns if Iterator.remove() is called multiple times per next().
 *
 * Iterator.remove() can be called only once per call to next().
 * http://docs.oracle.com/javase/6/docs/api/java/util/Iterator.html#remove%28%29
 *
 * This property warns if remove() is not preceded by next().
 *
 * @severity error
 */
```

```
Iterator_RemoveOnce(Iterator i) {
```

```
    event remove before(Iterator i) :
        call(void Iterator+.remove()) && target(i) {}
```

```
    event next before(Iterator i) :
        call(* Iterator+.next()) && target(i) {}
```

```
    ere : (next+ (remove | epsilon))*|
```

```
    @fail {
```

```
        RVMLogging.out.println(Level.CRITICAL, __DEFAULT_MESSAGE);
```

```
        RVMLogging.out.println(Level.CRITICAL, "Iterator.remove() can be called only once per call to next().");
```

```
    }
```

```
}
```

Novelty

1. **Specialize generating Regex by fine tuning our transformers on Extended Regex specific datasets**
2. **Improved accuracy**
3. **Rigorous and efficient semantic comparison with translator, allows testing on larger datasets**
4. **Open-source results**

Methods: Overview

NL

```
/**
 * Warns if Iterator.remove() is called multiple times per next().
 *
 * Iterator.remove() can be called only once per call to next().
 * http://docs.oracle.com/javase/6/docs/api/java/util/Iterator.html#remove%28%29
 *
 * This property warns if remove() is not preceded by next().
 *
 * @severity error
 */
```

Events (+the rest of the file)

```
Iterator_RemoveOnce(Iterator i) {

    event remove before(Iterator i) :
        call(void Iterator+.remove()) && target(i) {}

    event next before(Iterator i) :
        call(* Iterator+.next()) && target(i) {}
```

Blackbox
(Transformers, etc.)

Logic

ere : (next+ (remove | epsilon))*

Methods: LLMs to improve clarity of NL

- Our NL is vague compared to existing research (Req2LTL, nl2spec)

Ours: Prevents invocations of mark() if the Reader does not support.

Theirs: Globally, if the navigation system computes angles using dual inertial navigation modules, the input attitude angle shall be the weighted average of angles from module 1 and module 2 (weights 0.6 and 0.4), unless the system switches to single module or GPS signal is restored (from Req2LTL). [10]

- We use LLMs to improve clarity and give context:
 - When does the reader not support?
 - When is there an invocation of mark()?
 - What does the original language from Javadoc say?

Methods: LLMs to improve clarity of NL

- “Clarity of text” is subjective, and so we cannot quantitatively measure improvement of clarity
- Human annotations show that context and clarity is better

Old

Prevents invocations of mark() if the Reader does not support.

Context:

Mark Supported: Invocation of Reader.markSupported() that evaluates to false.

Mark Used: Invocation of Reader.mark(int readAheadLimit).

New

Structured Rule:

IF Mark Unsupported is true for a given Reader instance, THEN Mark Used MUST NOT occur on that instance.

`java.io.Reader.markSupported()`: "Tells whether this stream supports the mark() operation. The default implementation always returns false."

`java.io.Reader.mark(int)`: "Not all character-input streams support the mark() operation. [...] Throws: IOException - If the stream does not support mark(), or if some other I/O error occurs."

Methods: LLMs to improve clarity of event code

- LLMs are trained on far more NL than Code
- As context increases, performance of LLMs decreases
- Therefore, we are motivated to translate Code to NL

```
event remove before(Iterator i) :  
    call(void Iterator+.remove()) && target(i) {}  
  
event next before(Iterator i) :  
    call(* Iterator+.next()) && target(i) {}
```

Remove: The iterator i has .remove() called

Next: The iterator i has .next() called

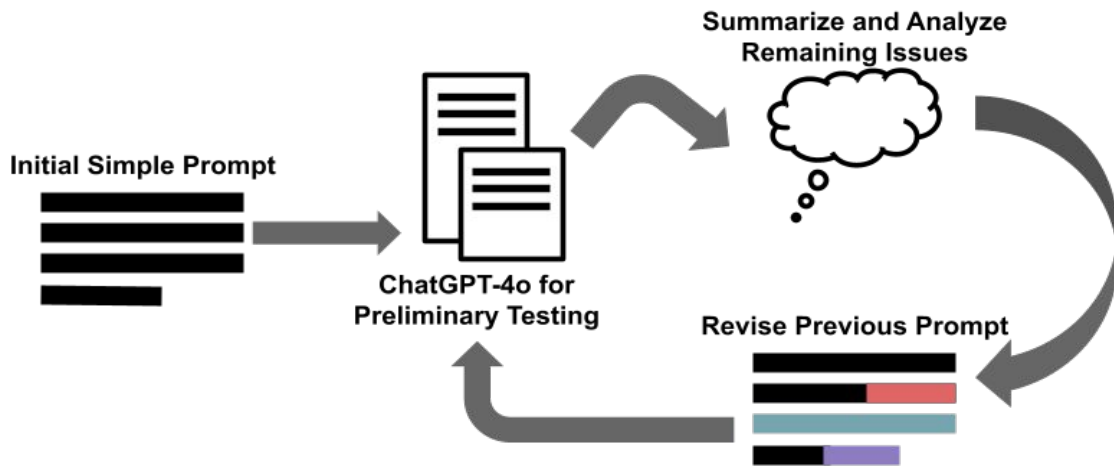
Methods: Fine Tuning LLMs to for Software Specifications

```
{
  "test_suite_id": 2,
  "regex": "-([a-z])",
  "project": {
    "id": 163842,
    "name": "@foal/swagger",
    "repo": "https://github.com/FoalTS/foal"
  },
  "positive_strings": [
    {
      "id": 7,
      "subject": "allow-same-version",
      "func": "RegExp#replace",
      "full_match": 0,
      "partial_match": 1,
      "first_sub_match_start": 5,
      "first_sub_match_end": 7
    }, ...]
  ],
  "negative_strings": [
    {
      "id": 25,
      "subject": "_auth",
      "func": "RegExp#replace",
      "full_match": 0,
      "partial_match": 0,
      "first_sub_match_start": -1,
      "first_sub_match_end": -1
    }, ...]
  ]
}
```

- Use dataset from Cakar et. al [2] with 55,137 regex composition tasks to fine-tune
- Tuned GPT-OSS-120b with Kiln [9]

- Results received 7/40 in testing
- The reasoning on local models is still too low
- Thus, we move forward continuing to use OpenAI 5.5 API

Methods: Prompt Development



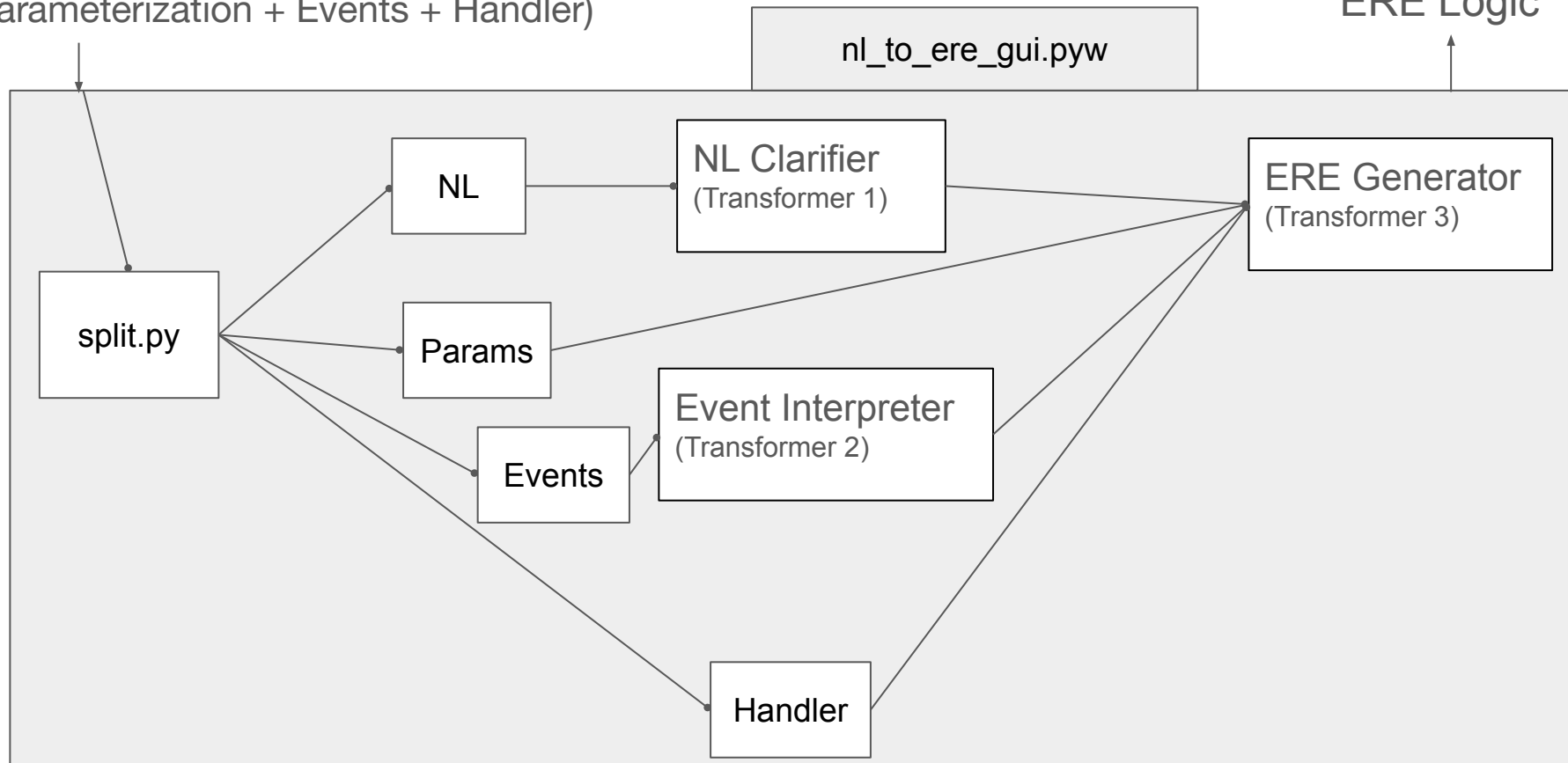
- We first use prompt engineering to for LLMs to generate our ERE
- Incorporate the guidelines for prompt engineering found at OpenAI [8]
- Extend insight for prompt prompt based on B. Çakar's study on Regex generation

Methods

	Iteration #0 (Accuracy 0/3)	Translate the following natural language into regular expression for java runtime-verification
Defines @fail and @match handlers. Provides ERE syntax dictionary. Elaborates more on characteristics of MOP (only events defined will be considered in the trace).	Iteration #1 (Accuracy 0/2)	The result of the regular expression will fall into two categories: match, meaning the expression matches the entire relevant trace, or fail, meaning the expression does not match onto the entire relevant trace. It should be indicated for the final expression which one of these results maps to a violation of the runtime property. Recall that regular expression uses "(" and ")" to group events, " " to signify "or", "*" to signify match as many as possible, and "+" to signify match as many as possible but at least once. For example, the expression "(A B)+" means match either event A or event B as many times as possible but at least once. In the regular expression, do not pass arguments to the events and only put the event name. Further, the expression will ignore any event not defined, so using "." is unnecessary.
Further elaborated on "only events defined will be considered in the trace." Elaborated on the logic that comes with "creation" events.	Iteration #2 (Accuracy 1/3)	Any events not named in the regular expression are automatically ignored and need not be accounted for using .*, other*, or any wildcard inside of the expression. The created regular expression should be as specific as possible and should begin from the creation of the object(s) in question.
Discourage usage of the ? operator and instead using epsilon.	Iteration #3 (Accuracy 1/3)	On the otherhand, operators like "?" and "{a,b}" do not exist and should not be used. For example, the expression "(A B)+" means match either event A or event B as many times as possible but at least once. Furthermore, this special form of Regex has added an event called "epsilon", which represents the event of nothing. For example, the expression "(A epsilon)" would match both the trace "A" and an empty trace.
Explicitly stated that all interactions should be considered in the ERE (A before B, B before A).	Iteration #4 (Accuracy 2/3)	To be able to match the relevant trace, analyze how the events in the natural language interact with each other at all times from creation. For example, even if the natural language specifies that event A cannot occur after event B, make sure to consider event A occurring before event B in order to consider the entire trace.
Explicitly stated that the trace can be unconventional, for example "shutdown" can be in the trace multiple times.	Iteration #5 (Accuracy 1/3)	In the trace, unconventional sequences events may occur, for example events like "terminate" and "start" could be ran multiple times. If the following natural language does not specify against it, cases like these need to be considered.

Methods: Full Architecture for Tool

File Without Formal Logic (NL +
Parameterization + Events + Handler)



nl_to_ere_gui.pyw

ERE Logic

Methods: Prepping ERE for Comparison

- First anonymize the ERE:

`create useiter* modify+ useiter => AB*C+B`

- **Creation events**

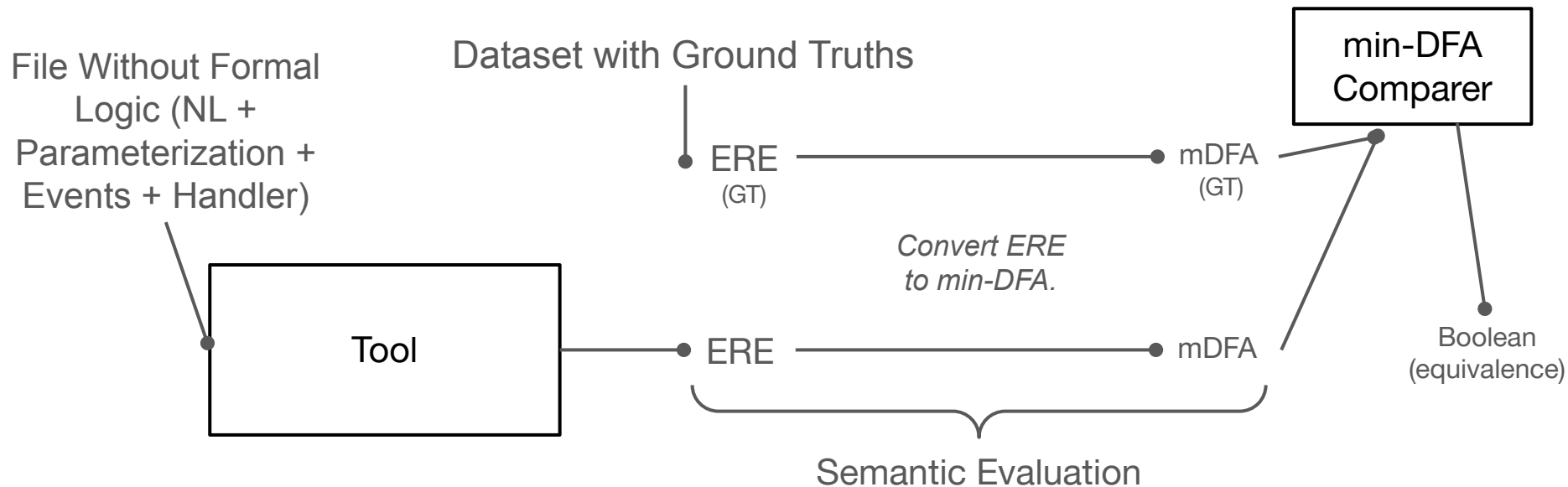
- This means that the specification **[A*]** (which without the creation event means match any amount of “A”) actually is the same meaning as **[A+]** (because when “A” is a creation event, it is required to happen at least once in the trace.)
- **A* = A+**
- **(A|epsilon)B = AB**

- **Terminating Events**

- The specification **[AB+]** which means “first a, then at least one b” in ERE has the same technical meaning as **[AB]**, which is just “first a, then one b.”
- **AB+ = AB**

- **Convert them into min-DFA [5], Then use existing min-DFA comparison tool [4].**

Methods: Validity Checking for a Specific Testing Pair



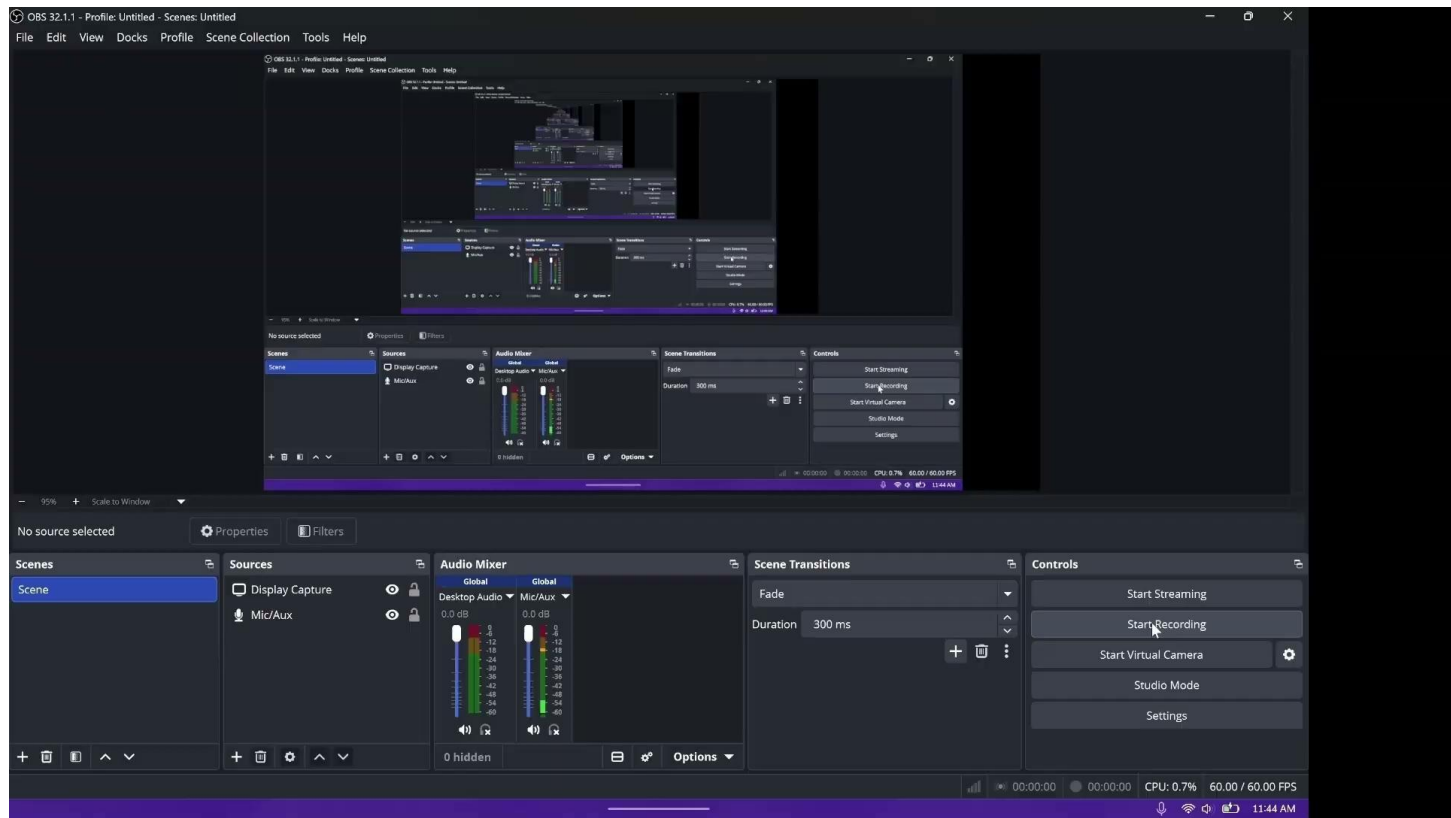
This is the general workflow for testing a transformer/prompt.

Results: Overview

Table 2: Performance of Evaluated Models on ERE Generation

Model Configuration	Semantic Accuracy	Syntactic Accuracy
GPT 5.5 w/ Iterative Prompt	0.42	0.32
GPT 5.5 w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.61	0.44
GPT-OSS w/ Iterative Prompt	0.22	0.15
GPT-OSS w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.28	0.19
Fine-tuned GPT-OSS w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.35	0.21

Results: Demo



Results: Specific Results

File	Generated	Groundtruth	Details
URLConnection_Connect	(implicit (explicit implicit)?)	ere : (explicit implicit) explicit+ @match	The question mark doesn't exist.
ProcessBuilder_NullKeyOrValue	createMap (nullPut nullQuery)+ Violate if the expression matches	ere : createMap (nullPut nullQuery)+ @match	
Socket_OutputStreamUnavailable	(Socket.<init>(Socket.connect(Socket.getOutputStream)*)?(Socket.shutdownOutput Socket.close)?)+	ere : (create_connected create_unconnected connect) get* (close shutdown)* @fail	Incorrect syntax
ShutdownHook_LateRegister	(register unregister)* start (register unregister) Failure => Violation	ere : (register unregister)* start+ (register unregister) @match	Does not account for many shutdowns, but potentially valid

Conclusion/Future Work

Automating generation of ERE will reduce barrier-of-entry into using RV tools and the runtime-verification tools which use RV tools [6, 7]

This project makes efficient and rigorous debugging more accessible to the average software engineer.

We hope to:

- Perform a survey of more LLMs
- Attempt to generate more forms of formal logic, beyond ERE
- Test results of using the tool in real industry setting rather than Javadoc specifications
- Attempt to utilize prior work and modify to fit our goals

References

- [1] M. Cosler, C. Hahn, D. Mendoza, F. Schmitt, and C. Trippel, “nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models,” arXiv.org, 2023. <https://arxiv.org/abs/2303.04864> (accessed Nov. 07, 2025).
- [2] B. Çakar, C. M. Sale, S. Chen, D. Lee, and J. C. Davis, “Is Reuse All You Need? A Systematic Comparison of Regular Expression Composition Strategies,” arXiv.org, 2025. <https://arxiv.org/abs/2503.20579>.
- [3] O. Legunsun, 2025, “property-db,” GitHub. [Online]. Available: <https://github.com/SoftEngResearch/tracemop/tree/3de76a81606fce70e893ab305320b310a614438a/scripts/props>.
- [4] Norton, Daphne, "Algorithms for testing equivalence of finite automata, with a grading tool for JFLAP" (2009). Thesis. Rochester Institute of Technology. Accessed from <https://repository.rit.edu/cgi/viewcontent.cgi?article=7944&context=theses>.
- [5] CyberZHG, Toolbox, 2023. <https://cyberzhg.github.io/toolbox/>
- [6] Feng Chen and Grigore Roşu. 2007. Mop: an efficient and generic runtime verification framework. SIGPLAN Not. 42, 10 (October 2007), 569–588. <https://doi.org/10.1145/1297105.1297069>
- [7] D. Jin, P. O. Meredith, C. Lee, and G. Rosu, “JavaMOP: Efficient parametric runtime monitoring framework,” *2013 35th International Conference on Software Engineering (ICSE)*, pp. 1427–1430, Jun. 2012, doi: <https://doi.org/10.1109/icse.2012.6227231>.
- [8] OpenAI, “Prompt engineering,” openai.com, 2026. <https://platform.openai.com/docs/guides/prompt-engineering>
- [9] A. Lewis and L. Radford, “Kiln,” github.com, 2026. <https://github.com/Kiln-AI/Kiln>
- [10] Z. Ma, C. Wen, Z. Su, X. Liang, C. Tian, and S. Qin, "Bridging Natural Language and Formal Specification—Automated Translation of Software Requirements to LTL via Hierarchical Semantics Decomposition Using LLMs," 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), Seoul, Korea, Republic of, 2025, pp. 1208-1220, doi: 10.1109/ASE63991.2025.00104.